CHEMINS DE POIDS MINIMUM

F. M.

2022/2023

SOMMAIRE

1. GRAPHE PONDÉRE
2. CHEMIN DE POIDS MINIMUM
3. PROPRIÉTÉ FONDAMENTALE
4. LES DIFFÉRENTS PROBLÈMES
5. ALGORITHME DE BASE
6. LES DIFFÉRENTS ALGORITHMES

Définition 1.1

Un **graphe pondéré** est un triplet (S, A, P) où :

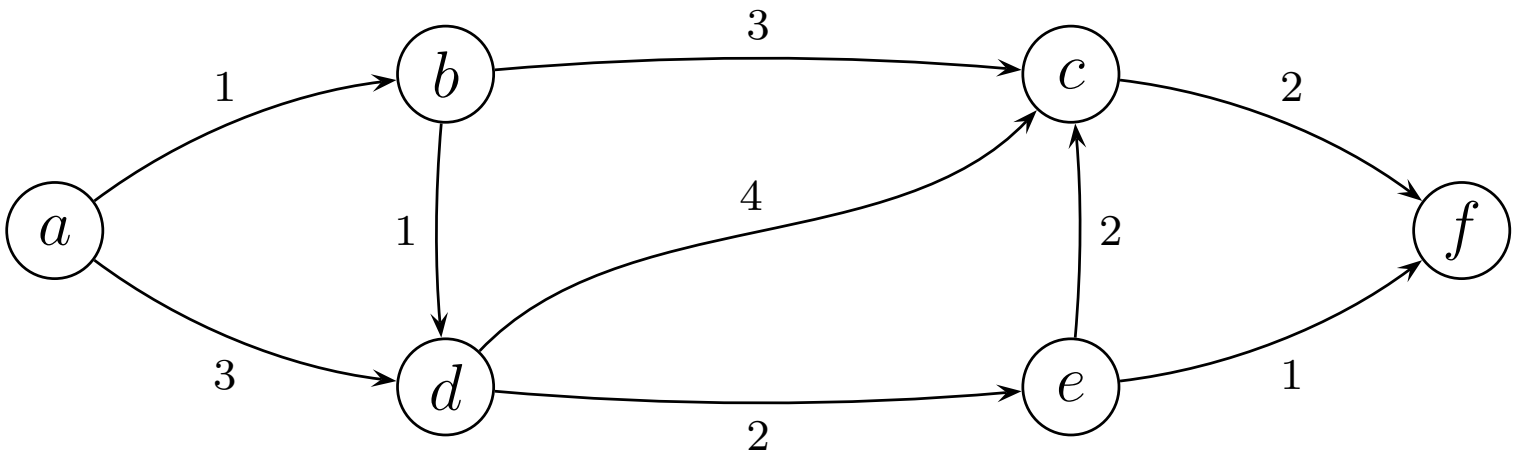
- S est un ensemble fini de sommets,
- A est un ensemble d'arcs (ou d'arêtes),
- P est une application appelée **pondération** qui à tout arc (ou arête) $u \in A$ associe le nombre $P(u)$ appelé **poids** de u.

Remarques

Les poids peuvent représenter des distances, des durées, des coûts, des capacités, des probabilités, etc.

Dans ce chapitre, le terme graphe désigne un graphe orienté, simple, connexe et pondéré.

Diagramme d'un graphe pondéré



$P(a,b) = 1$ $\quad$ $P(a,d) = 3$

$P(b,c) = 3$ $\quad$ $P(b,d) = 1$

$P(c,f) = 2$

$P(d,c) = 4$ $\quad$ $P(d,e) = 2$

$P(e,c) = 2$ $\quad$ $P(e,f) = 1$

Définition 1.2

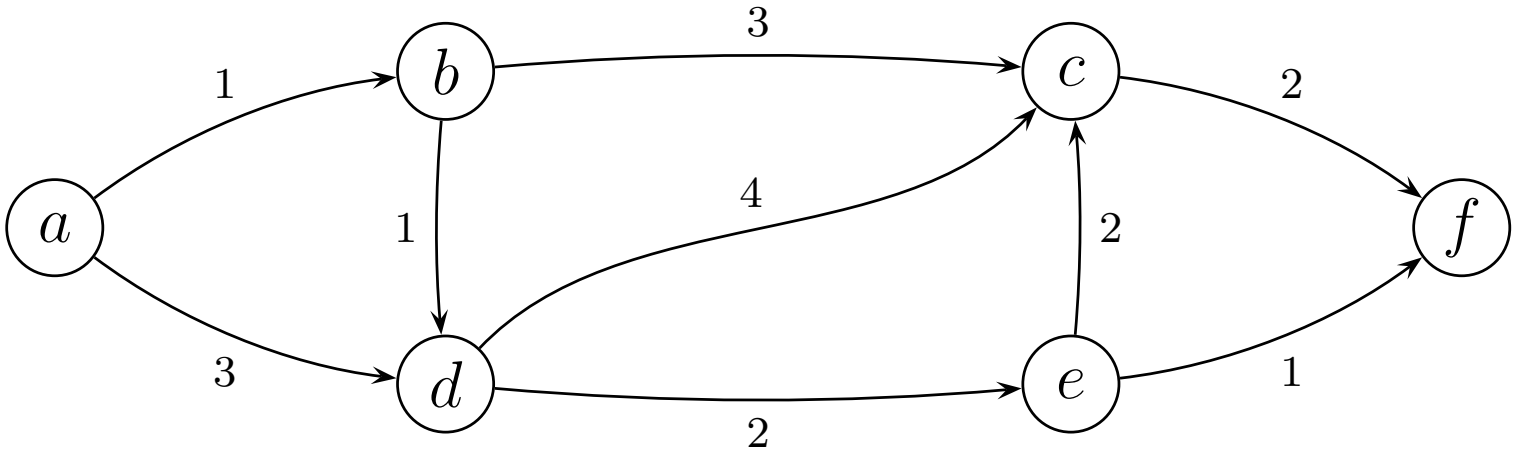
Soit $G = (S, A, P)$ un graphe pondéré. On définit le **poids** d'un chemin dans G comme la somme des poids des arcs qui composent ce chemin. On note $P(s_0, s_1, \ldots, s_k)$ le poids du chemin $(s_0, s_1, \ldots, s_k)$ et :

$$\boldsymbol{P(s_0, s_1, \ldots, s_k) = \sum_{i=1}^{k} P(s_{i-1}, s_i)}$$

Remarques

- Si $P(u) = 1$ pour chaque arc $u \in A$ alors le poids d'un chemin est égal à sa longueur.
- On convient que le poids d'un chemin de longueur 0 est nul.

Illustration



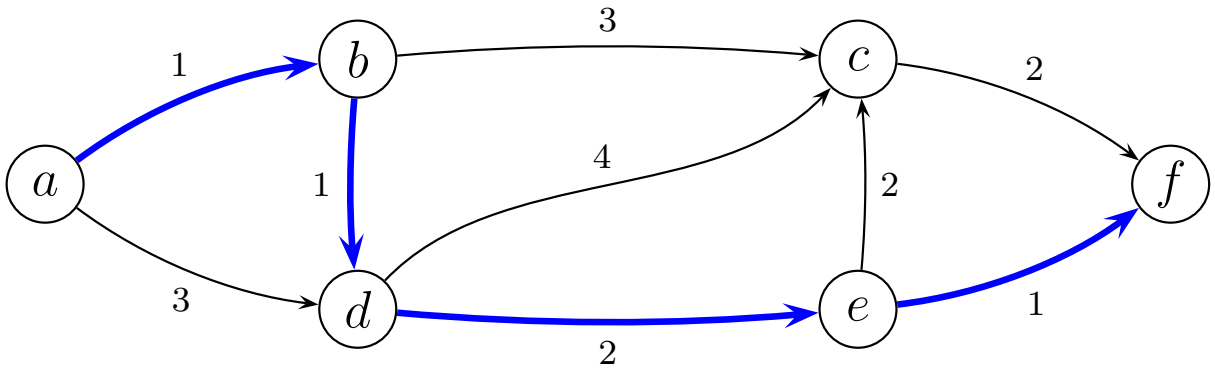
$$P(a,b,d) = P(a,b) + P(b,d) = 2 \qquad \ell(a,b,d) = 2$$

$$P(a,b,d,c) = P(a,b) + P(b,d) + P(d,c) = 6 \qquad \ell(a,b,d,c) = 3$$

$$P(a,b,d,e,f) = 5 \qquad \ell(a,d,b,e,f) = 4$$

Définition 2.1

Un chemin C allant de x à y dans un graphe G est un chemin de **poids minimum** (**cpm** en abrégé) si et seulement si tout autre chemin allant de x à y dans G a un poids supérieur au égal à celui de C.



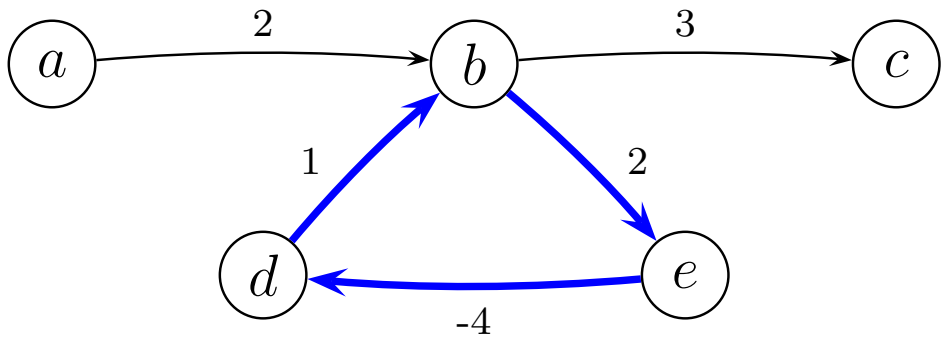
$$P(a,d,c,f) = 9 \quad P(a,d,e,c,f) = 9 \quad P(a,b,d,c,f) = 8 \quad P(a,b,d,e,c,f) = 8$$

$$P(a,b,c,f) = 6 \quad P(a,d,e,f) = 6 \quad P(a,b,d,e,f) = 5$$

(a,b,d,e,f) est un cpm allant de a à f et $\textsc{Dist}(a,f) = 5$

Définition 2.2

Un circuit de poids négatif est dit **absorbant**.



$P(b, e, d, b) = -1$
(b, e, d, b) est un circuit absorbant de poids -1

$$P(a, b, c) = 5$$

$$P(a, b, e, d, b, c) = 5 - 1 = 4$$

$$P(a, b[, e, d, b]^{\times k}, c) = 5 - k$$

$$\lim_{k \to +\infty} P(a, b[, e, d, b]^{\times k}, c) = -\infty$$

Conséquence

Si un sommet s de G appartient à un circuit absorbant alors il n'existe aucun chemin de poids minimum passant par s.

Définition 2.3

Soit $\mathcal{C}$ un chemin de poids minimum allant d'un sommet x à un sommet y dans un graphe G. Alors $P(C)$ est appelé distance de x à y dans G. On note $\textsc{Dist}(x, y)$ la distance de x à y dans G.

Deux cas particuliers :

- $\textsc{Dist}(x, y) = \infty$: il n'existe aucun chemin allant de x à y dans G.
- $\textsc{Dist}(x, y)$ non définie : il existe un chemin de x à y dans G contenant un circuit absorbant.

3. PROPRIÉTÉ FONDAMENTALE

Propriété d'optimalité

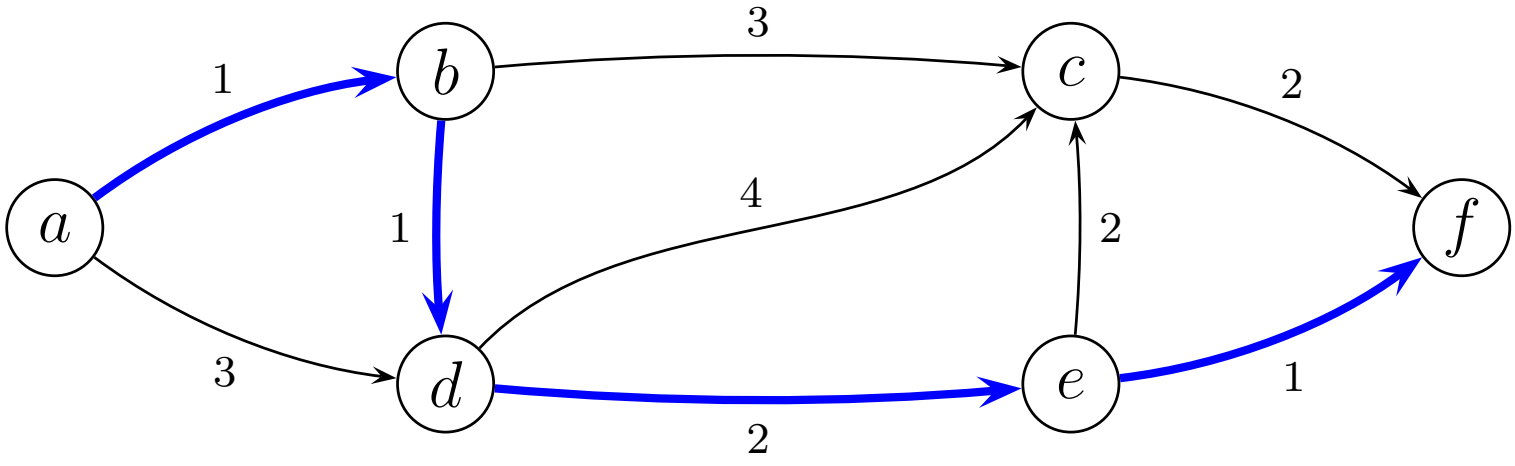
Soit G un graphe sans circuit absorbant. Alors, **tout sous-chemin d'un chemin de poids minimum est un chemin de poids minimum**.

Soit $C(s \rightsquigarrow p)$ un cpm allant du sommet s au sommet p dans G. Soit $C(x \rightsquigarrow y)$ un sous-chemin de $C(s \rightsquigarrow p)$ allant du sommet x au sommet y.

Hypothèse : $C(x \rightsquigarrow y)$ n'est pas un cpm allant de x à y.

Il existe alors un chemin $C'(x \rightsquigarrow y)$ dans G dont le poids est inférieur à celui de $C(x \rightsquigarrow y)$. Si on substitue $C'(x \rightsquigarrow y)$ à $C(x \rightsquigarrow y)$ dans le chemin $C(s \rightsquigarrow p)$, on obtient alors un nouveau chemin de s à p dont le poids est inférieur à celui de $C(s \rightsquigarrow p)$, ce qui est impossible. L'hypothèse est donc fausse.

Illustration



(a, b, d, e, f) est un cpm de a à f

(a, b), (a, b, d), (a, b, d, e), (b, d), (b, d, e), (b, d, e, f), (d, e), (d, e, f) et (e, f) sont aussi des cpm

Problème 1

Trouver un chemin de poids minimum allant d'un sommet x vers un sommet y dans un graphe pondéré G. Problème équivalent : déterminer la distance de x à y dans G.

Trois situations sont possibles

- Si G contient un circuit absorbant alors il est possible que le problème n'ait pas de solution.
- Si G est sans circuit absorbant mais ne contient aucun chemin allant de x à y alors le problème n'a pas de solution.
- Si G est sans circuit absorbant et contient des chemins allant de x à y alors il existe une solution à ce problème.

Problème 2

Soit r un sommet d'un graphe pondéré G. Pour chaque descendant s de r, trouver un chemin de poids minimum allant de r à s dans G. Problème équivalent : pour tout sommet s de G, déterminer $\textsc{Dist}(r, s)$.

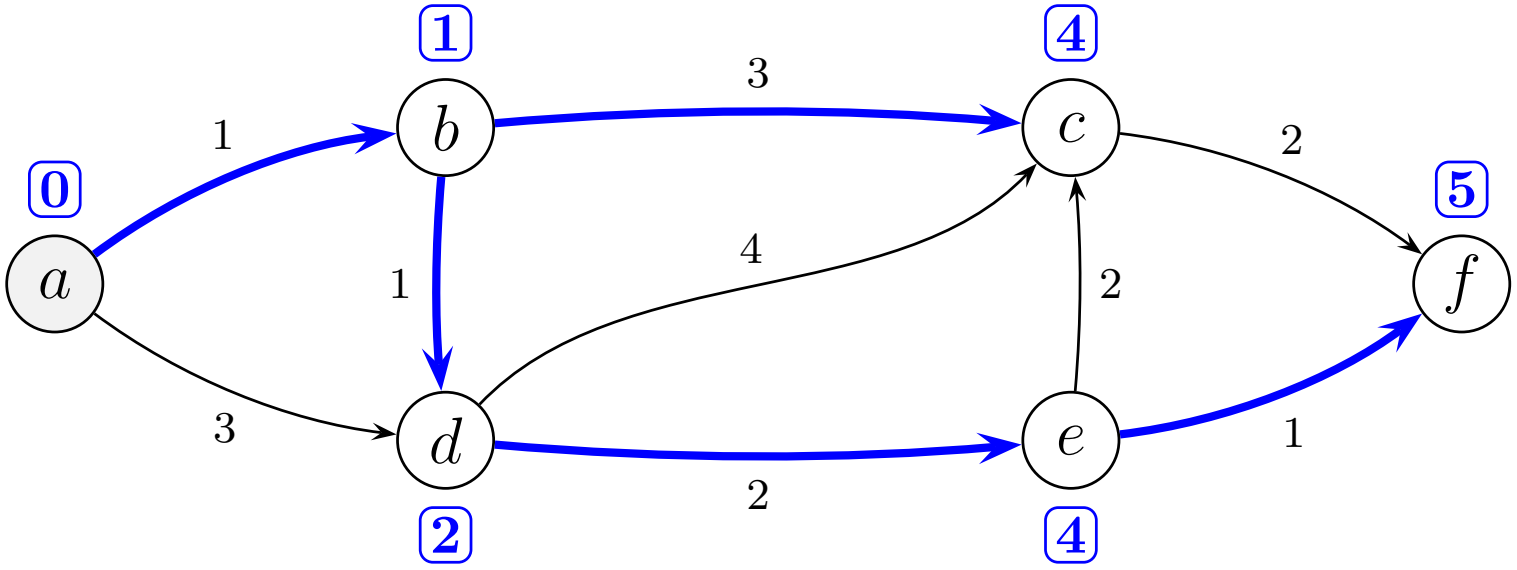
Propriété 4.1 – Conséquence de la propriété d'optimalité

Soit G un graphe pondéré sans circuit absorbant. Alors G contient une r-arborescence formée de chemins de poids minimum allant de r vers chacun des descendants de r. Cette arborescence est appelée **r-arborescence de chemins de poids minimum**.

Remarque

Le Problème 2 permet de résoudre le Problème 1. En fait, il est plus *facile* de résoudre le Problème 2 que le Problème 1.

Arborescence de cpm



(a, b) est un cpm de a à b
(a, b, c) est un cpm de a à c
(a, b, d) est un cpm de a à d
(a, b, d, e) est un cpm de a à e
(a, b, d, e, f) est un cpm de a à f

a-arborescence de chemins de poids minimum

annotation d'un sommet s : $\boxed{\textsc{Dist}(a,s)}$

Problème 3

Pour chaque couple (x, y) de sommets d'un graphe pondéré G, trouver un chemin de poids minimum allant de x à y. Problème équivalent : déterminer $\textsc{Dist}(x, y)$ pour chaque couple (x, y) de sommets de G.

Remarque

Le Problème 2 permet de trouver, pour chaque sommet r de G, une r-arborescence de chemins de poids minimum. Le Problème 2 permet donc de résoudre le Problème 3. Il est cependant plus *avantageux* de résoudre directement le Problème 3.

5. ALGORITHME DE BASE

Objectif

Soit $G = (S, A, P)$ un graphe pondéré **sans circuit absorbant** et soit r un sommet de G. L'algorithme de base détermine **une r-arborescence de chemins de poids minimum** dans G.

Représentation d'une r-arborescence

On utilise deux tables. Une **table des pères $\mathcal{A}$** pour mémoriser la structure de l'arborescence. Une **table des potentiels Π** pour mémoriser les poids des chemins d'origine r dans l'arborescence.

Procédure fondamentale

L'algorithme de base utilise une procédure fondamentale appelée **procédure de relâchement**. Cette procédure est utilisée par tous les algorithmes de recherche de chemins de poids minimum.

Table des pères et table des potentiels

Pour représenter une r-arborescence T incluse dans G, on utilise :

- Une **table des pères**, définie comme suit :

$$\forall s \in S,\ \mathcal{A}[s] = \begin{cases} \text{none} & \text{si } s = r \\ x & \text{si l'arc } (x, s) \in T \\ s & \text{si } s \notin T \end{cases}$$

- Une **table des potentiels**, définie comme suit :

$$\forall s \in S,\ \Pi[s] = \begin{cases} \text{poids du chemin de } r \text{ à } s \text{ dans } T & \text{si } s \in T \\ \infty & \text{si } s \notin T \end{cases}$$

Le nombre $\mathbf{\Pi[s]}$ est appelé **potentiel** du sommet s.

Propriétés de la table des potentiels

Soit Π la table des potentiels associée à une r-arborescence T incluse dans G. On a :

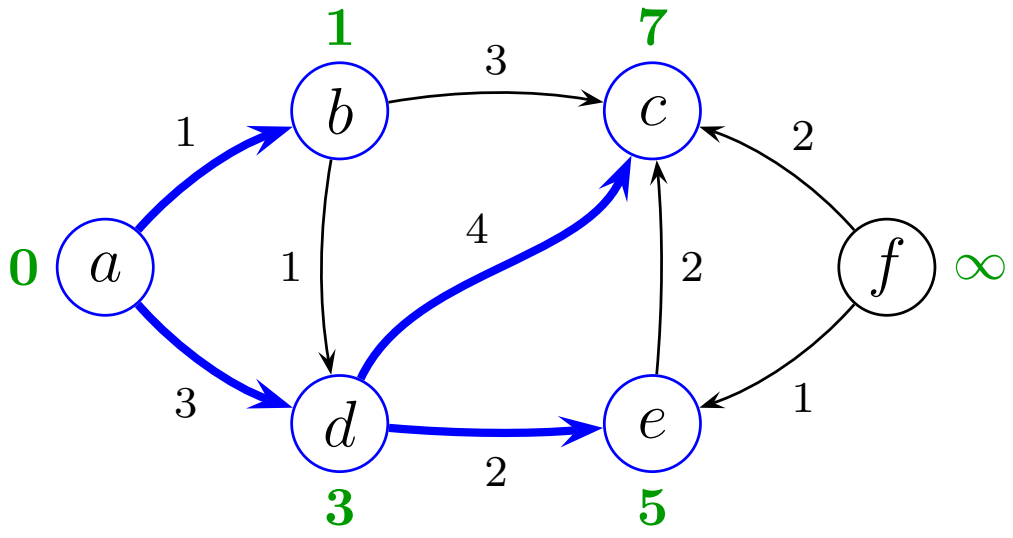
- $\boldsymbol{\Pi[r] = 0}$
- $\boldsymbol{\Pi[x] + P(x,y) = \Pi[y]}$, pour tout arc (x, y) de T.
- $\boldsymbol{\Pi[x] \geq \textsc{Dist}(r,x)}$, pout tout sommet x de G. On dit que $\Pi[x]$ est un majorant de $\textsc{Dist}(r, x)$.

Remarque

Si une r-arborescence qui ne contient pas tous les descendants de r dans G est dite **partielle**.

Illustration

T, une a-arborescence :



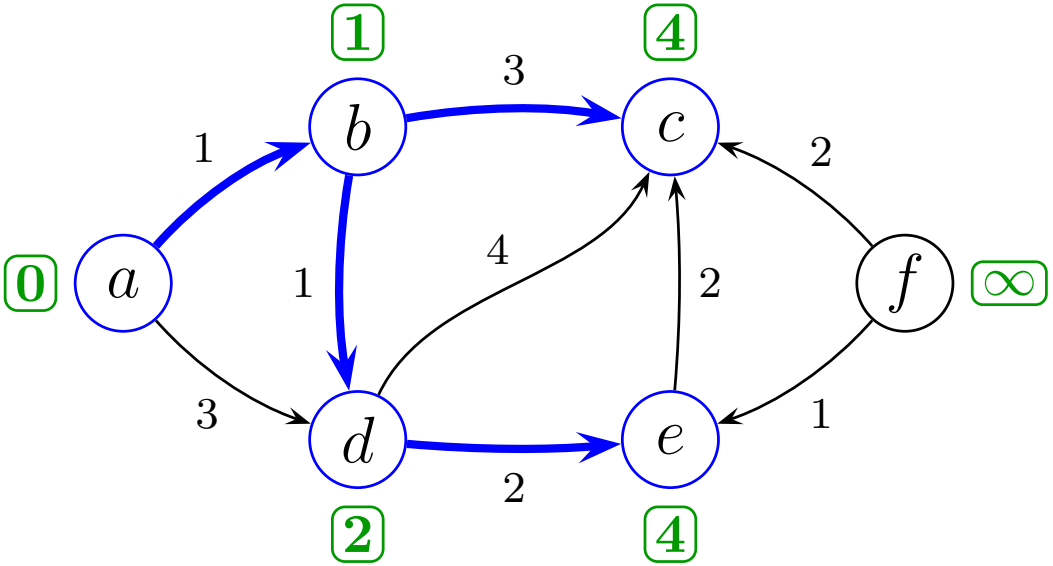
s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	a	d	a	d	f
$\Pi[s]$	0	1	7	3	5	∞

pour tout arc (x, y) de T, $\Pi[x] + P(x, y) = \Pi[y]$

$$\forall s \in S,\ \Pi[s] \geq \textsc{Dist}(a, s)$$

Illustration

T, une a-arborescence de cpm :

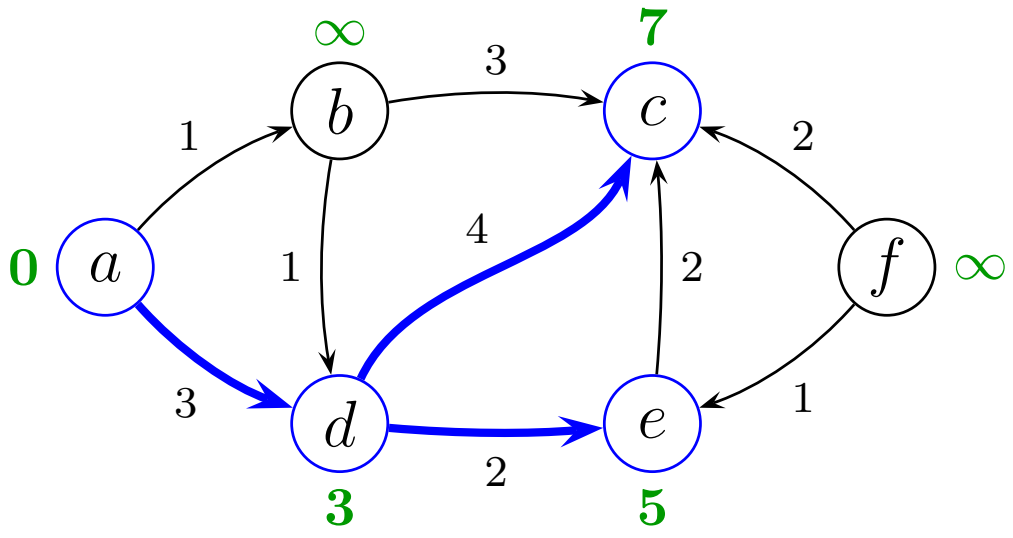


s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	a	b	b	d	f
$\Pi[s]$	0	1	4	2	4	∞

$$\forall s \in S,\ \Pi[s] \ = \ \textsc{Dist}(a, s)$$

Illustration

T, un a-arborescence partielle :



s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	b	d	a	d	f
$\Pi[s]$	0	∞	7	3	5	∞

$$\forall s \in S,\ \Pi[s] \ \geq\ \textsc{Dist}(a, s)$$

Propriété 5.1

Une r-arborescence complète T incluse dans un graphe $G = (S, A)$ est une r-arborescence de cpm si et seulement si :

$$\forall (x, y) \in A,\ \Pi[x] + P(x, y) \geq \Pi[y]$$

($\Rightarrow$) Si T est une r-arborescence de cpm, les inégalités sont vérifiées (d'après la propriété d'optimalité).
($\Leftarrow$) Soit T une r-arborescence complète vérifiant les inégalités. Considérons un chemin $\mathcal{C} = (r, s_1, \ldots, s_k, s)$ allant de r à s dans G. Alors les arcs de $\mathcal{C}$ vérifient :

$$\begin{aligned}
\Pi[r] + P(r, s_1) &\geq \Pi[s_1] \\
\Pi[s_1] + P(s_1, s_2) &\geq \Pi[s_2] \\
\ldots &\geq \ldots \\
\Pi[s_k] + P(s_k, s) &\geq \Pi[s]
\end{aligned}$$

La somme de ces inégalités donne : $\Pi[r] + P(\mathcal{C}) \geq \Pi[s]$. Or $\Pi[r] = 0$, d'où : $P(\mathcal{C}) \geq \Pi[s]$. Le chemin de r à s dans T est donc un cpm de r à s dans G.

Relâchement d'un arc

Soient Π et $\mathcal{A}$ les tables représentant une r-arborescence T. La procédure de **relâchement** d'un arc (s, x) de G est la suivante :

proc relacher(x, y)
  si $\Pi[s] + P(s, x) < \Pi[x]$ **alors**
    $\Pi[x] \leftarrow \Pi[s] + P(s, x)$
    $\mathcal{A}[x] \leftarrow s$
  fin si
fin proc

$$\Leftrightarrow \boldsymbol{\Pi[x] \leftarrow \min\{\Pi[x], \Pi[s] + P(s, x)\}}$$

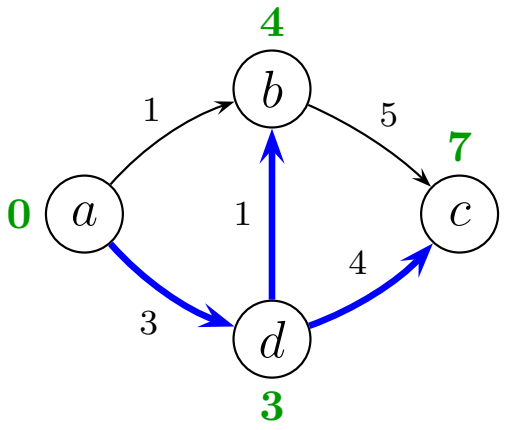
Un arc (s, x) vérifiant $\Pi[s] + P(s, x) < \Pi[x]$ est dit **améliorant**.

$\Pi[x]$ est le poids du chemin de T allant de r à x. $\Pi[s] + P(s, x)$ est le poids du chemin de T allant r à s prolongé par l'arc (s, x). La procédure retient le meilleur de ces deux chemins et modifie T en conséquence.

Le relâchement d'un arc améliore ponctuellement l'arborescence.

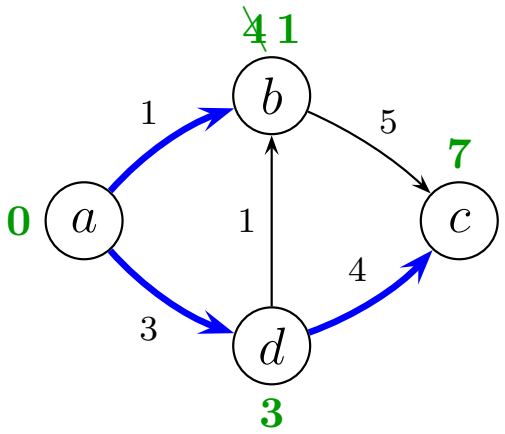
Illustration

s	a	b	c	d
$\mathcal{A}[s]$	none	d	d	a
$\Pi[s]$	0	4	7	3



L'arc (a, b) est améliorant : $\Pi[a] + P(a, b) = 1 < \Pi[b] = 4$
$\Rightarrow$ relâcher (a, b) améliore l'arborescence : $\Pi[b] \leftarrow 1$ et $\mathcal{A}[b] \leftarrow a$.

s	a	b	c	d
$\mathcal{A}[s]$	none	$\cancel{d}$ a	d	a
$\Pi[s]$	0	$\cancel{4}$ 1	7	3



Algorithme : Initialisation des tables

pour tout $s \in S$ **faire**
  $\Pi[s] \leftarrow \infty$; $\mathcal{A}[s] \leftarrow s$
fin pour
$\Pi[r] \leftarrow 0$; $\mathcal{A}[r] \leftarrow$ none

Commentaires

Le point de départ de l'algorithme est une arborescence triviale réduite au seul sommet r. Si le graphe ne contient pas de circuit absorbant, les valeurs $\Pi[r] = 0$ et $\mathcal{A}[r] =$ none ne pourront pas être améliorées.

Le potentiel de chaque sommet $s \neq r$ est fixé à ∞. C'est une *estimation pessimiste* de la distance de r à s qui vérifie $\Pi[s] \geq \textsc{Dist}(r, s)$.

Algorithme : les itérations

Tant qu'il existe un arc améliorant (s, x) dans G faire : relacher(x, y).

Commentaires

À chaque itération, un arc améliorant est relâché et les tables $\mathcal{A}$ et Π sont mises à jour. L'itération est répétée jusqu'à ce que le graphe ne contienne plus aucun arc améliorant.

L'arrêt de l'algorithme se produit quand $\Pi[s] + P(s, x) \geq \Pi[x]$, pour chaque arc (s, x) du graphe. La propriété 4.1 permet alors de conclure que l'arborescence obtenue par l'algorithme est bien une r-arborescence de cpm.

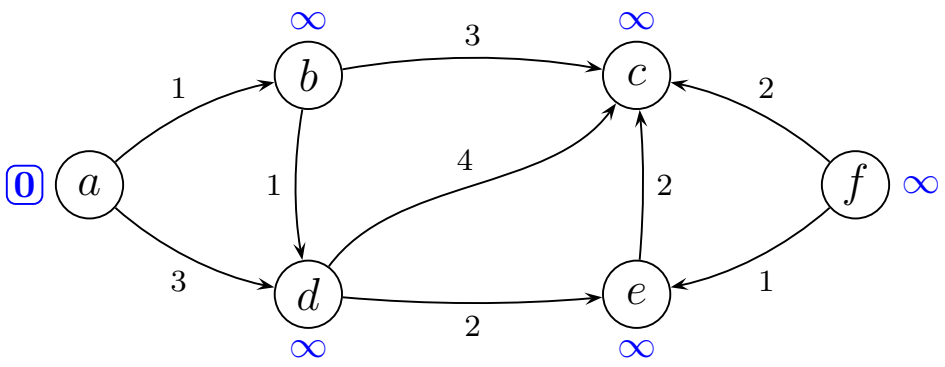
La terminaison de l'algorithme est garantie par l'absence de circuits absorbants dans le graphe.

Attention : cet algorithme ne fonctionnera pas si le graphe contient des circuits absorbants (boucle infinie), à moins de les détecter.

Illustration

ité. 0

initialisation quand $r = a$



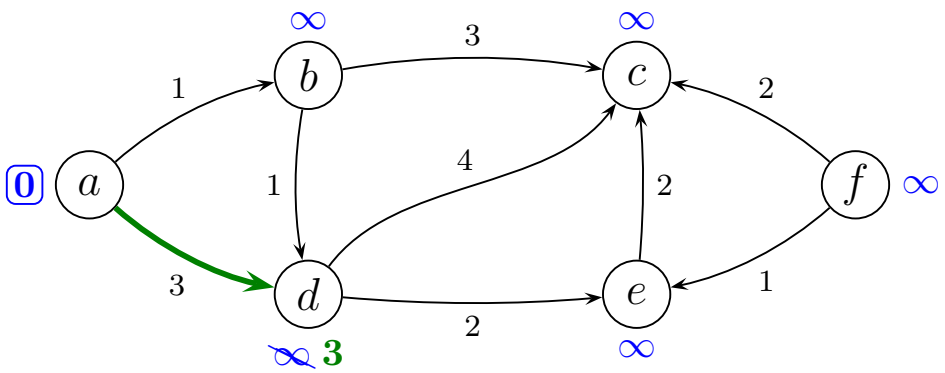
relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f

À chaque itération on ajoute une ligne au tableau des résultats. Cette ligne contient la représentation de l'arborescence obtenue en fin d'itération.

Illustration

ité. 1

relâchement de l'arc (a, d)



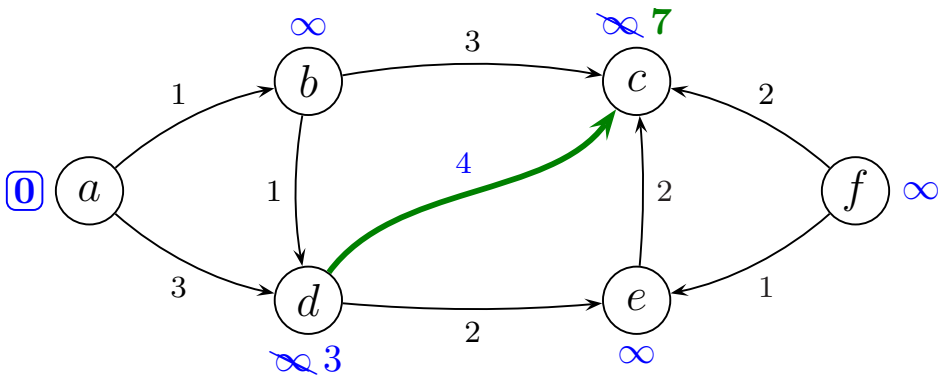
relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$\mathbf{3, a}$	∞, e	∞, f

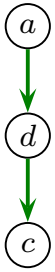
À chaque relâchement d'un arc améliorant (s, x), on modifie la marque du sommet x sur le diagramme et on ajoute une ligne au tableau des résultats.

Illustration

ité. 2

relâchement de l'arc (d, c)



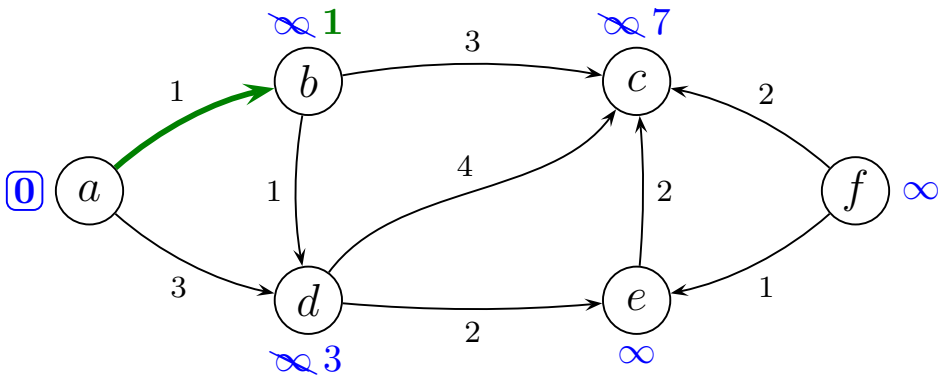


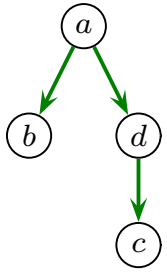
relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$3, a$	∞, e	∞, f
(d, c)	0, none	∞, b	$\mathbf{7, d}$	$3, a$	∞, e	∞, f

Illustration

ité. 3

relâchement de l'arc (a, b)



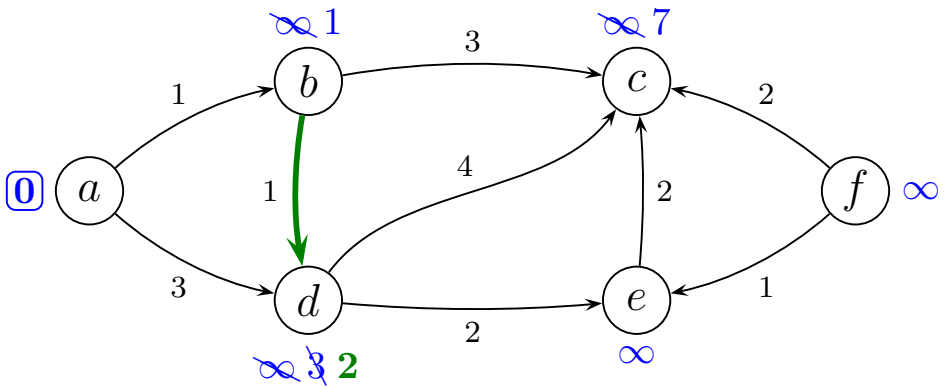


relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$3, a$	∞, e	∞, f
(d, c)	0, none	∞, b	$7, d$	$3, a$	∞, e	∞, f
(a, b)	0, none	$\mathbf{1, a}$	$7, d$	$3, a$	∞, e	∞, f

Illustration

ité. 4

relâchement de l'arc (b, d)

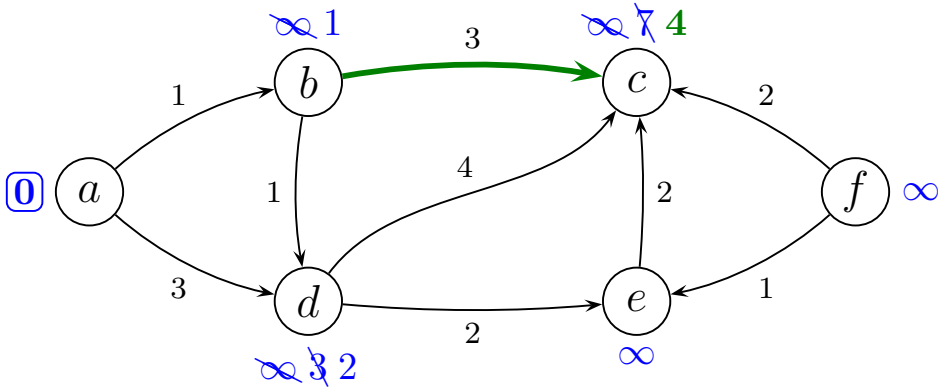


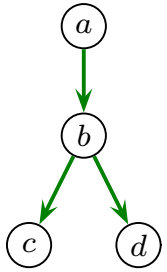
relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$3, a$	∞, e	∞, f
(d, c)	0, none	∞, b	$7, d$	$3, a$	∞, e	∞, f
(a, b)	0, none	$1, a$	$7, d$	$3, a$	∞, e	∞, f
(b, d)	0, none	$1, a$	$7, d$	$\mathbf{2, b}$	∞, e	∞, f

Illustration

ité. 5

relâchement de l'arc (b, c)



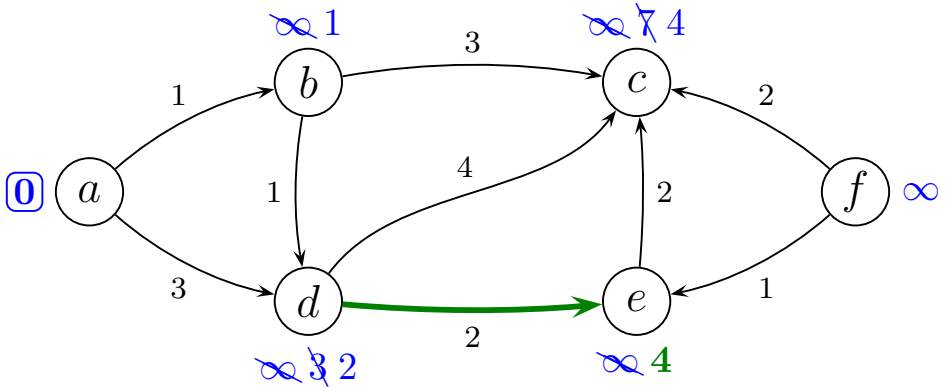


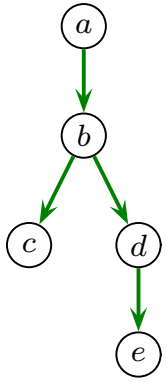
relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$3, a$	∞, e	∞, f
(d, c)	0, none	∞, b	$7, d$	$3, a$	∞, e	∞, f
(a, b)	0, none	$1, a$	$7, d$	$3, a$	∞, e	∞, f
(b, d)	0, none	$1, a$	$7, d$	$2, b$	∞, e	∞, f
(b, c)	0, none	$1, a$	$\mathbf{4, b}$	$2, b$	∞, e	∞, f

Illustration

ité. 6

relâchement de l'arc (d, e)



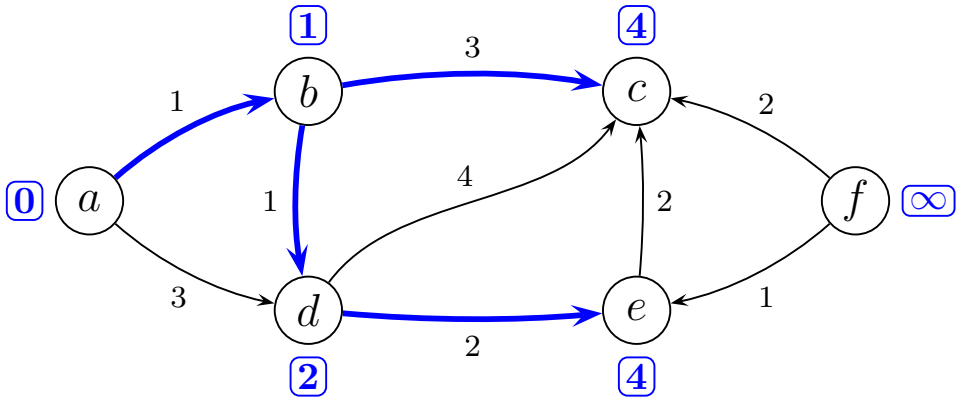


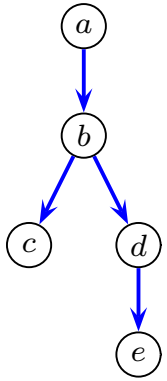
relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$3, a$	∞, e	∞, f
(d, c)	0, none	∞, b	$7, d$	$3, a$	∞, e	∞, f
(a, b)	0, none	$1, a$	$7, d$	$3, a$	∞, e	∞, f
(b, d)	0, none	$1, a$	$7, d$	$2, b$	∞, e	∞, f
(b, c)	0, none	$1, a$	$4, b$	$2, b$	∞, e	∞, f
(d, e)	0, none	$1, a$	$4, b$	$2, b$	$\boldsymbol{4, d}$	∞, f

Illustration

fin

il n'y a plus d'arc améliorant ⇒ fin de l'algorithme





relâcher	a	b	c	d	e	f
–	0, none	∞, b	∞, c	∞, d	∞, e	∞, f
(a, d)	0, none	∞, b	∞, c	$3, a$	∞, e	∞, f
(d, c)	0, none	∞, b	$7, d$	$3, a$	∞, e	∞, f
(a, b)	0, none	$1, a$	$7, d$	$3, a$	∞, e	∞, f
(b, d)	0, none	$1, a$	$7, d$	$2, b$	∞, e	∞, f
(b, c)	0, none	$1, a$	$4, b$	$2, b$	∞, e	∞, f
(d, e)	0, none	$1, a$	$4, b$	$2, b$	$4, d$	∞, f
résultat	**0, none**	**$1, a$**	**$4, b$**	**$2, b$**	**$4, d$**	**∞, f**

Algorithme de FORD

L'algorithme de Ford est une implémentation directe de l'algorithme de base qui détecte les circuits absorbants.

On appelle **phase de relâchement** le fait de relâcher tous les arcs du graphe. L'algorithme effectue un **maximum de n répétitions** de la phase de relâchement où n est le nombre de sommets du graphe. Si l'arborescence n'est pas améliorée au cours d'une phase, c'est qu'elle est optimale et l'algorithme est arrêté. Si la n^{e} phase améliore des potentiels, c'est que le graphe contient des circuits absorbant.

Considérons un chemin $\mathcal{C}$ de k arcs. Il est possible de calculer le poids de $\mathcal{C}$ si les arcs de $\mathcal{C}$ sont relâchés dans le sens de parcours. Mais si les arcs son relâchés dans le sens inverse du parcours, il faudra alors effectuer k phases.

Dans un graphe d'ordre n, tout chemin élémentaire est de longueur au plus $n-1$ arcs. Il faudra donc effectuer au maximum $n-1$. La n^{e} phase ne se produira que si le graphe contient un chemin qui est hamiltonien et de poids minimum ou si le graphe contient des circuits absorbants.

Algorithme de BELLMAN

L'algorithme de Bellman permet de déterminer une r-arborescence de cpm dans un graphe pondéré sans circuit (donc sans circuit absorbant).

Supposons que G soit un DAG d'ordre n admettant une unique source r et soit $(s_1 = r, s_2, \ldots, s_n)$ un ordre topologique pour G. On obtient alors une r-arborescence de cpm en procédant comme suit :

$\Pi[r] \leftarrow 0$
$\mathcal{A}[r] \leftarrow$ none
pour tout $i \leftarrow 2$ ***à*** n **faire**
  relâcher tous les arcs de but s_i
fin pour

À la i^{e} itération, $2 \leq i \leq n$, les cpm allant de r vers les prédécesseurs de s_i sont connus. En relâchant tous les arcs de but s_i, on obtient alors un cpm allant de r à s_i.

Algorithme de DIJKSTRA

L'algorithme de Dijkstra permet de déterminer une r-arborescence de cpm dans un graphe à pondération positive (donc sans circuit absorbant).

L'algorithme maintient un ensemble E contenant des sommets dits *examinés* : un sommet x est dit examiné quand $\Pi[x] = \textsc{Dist}(r, x)$. Le principe de l'algorithme est le suivant :

(i) Initialisation : $\Pi[r] \leftarrow 0$, $\mathcal{A}[r] \leftarrow$ none, $E \leftarrow \{r\}$ et $s \leftarrow r$.

(ii) Relâcher tous les arcs d'origine s.

(iii) Rechercher le sommet de plus petit potentiel n'appartenant pas à E. Si x est ce sommet alors : $\Pi[x] = \textsc{Dist}(r, x)$. Ce résultat est une conséquence de la positivité de la pondération.

(iv) Ajouter x à E, poser $s \leftarrow x$ et retourner en (ii).

Algorithme de FLOYD

L'algorithme de Floyd détermine pour chaque couple de sommets (x, y) d'un graphe pondéré, un cpm allant de x à y. Il détecte aussi la présence de circuits absorbants dans le graphe.

Plus précisément, cet algorithme détermine une s-arborescence de cpm pour chaque sommet s du graphe. La table des pères et la table des potentiels sont alors des tables à deux dimensions (des matrices).

Dans cet algorithme, un sommet est traité à chaque itération. À l'itération s et pour chaque couple de sommets (x, y), on regarde si un meilleur chemin allant de x à y peut être obtenu en concaténant un chemin de x à s et un chemin de s à y. Si oui, les tables sont mises à jour en conséquence.